- Functions implemented in the context of Spring Cloud Function can be exposed and integrated with external systems in different ways.
- Functions can be exposed as REST endpoints, packaged, and deployed in a FaaS platform as serverless applications (Knative, AWS Lambda, Azure Function, Google Cloud Functions), or they can be bound to message channels.
- Spring Cloud Stream, built on top of Spring Cloud Function, provides you with all the necessary plumbing to integrate your functions with external messaging systems like RabbitMQ or Kafka.
- Once you implement your functions, you don’t have to make any changes to your code. You only need to add a dependency on Spring Cloud Stream and configure it to adapt to your needs.
- In Spring Cloud Stream, destination binders provide integration with external messaging systems.
- In Spring Cloud Stream, destination bindings (input and output) bridge the producers and consumers in your applications with exchanges and queues in a message broker like RabbitMQ.
- Functions and consumers are activated automatically when new messages arrive.
- Suppliers need to be explicitly activated, such as by explicitly sending a message to a destination binding.

---

[1] See Sam Newman, *Monolith to Microservices* (O’Reilly, 2019).